Volcano for the Agentic AI Era:

Unified Scheduling Across Training, Inference, and Agents

Ricardo Aravena · Rainie Li

KubeCon + CloudNativeCon North America 2026
Salt Lake City · November 9–12

Speakers

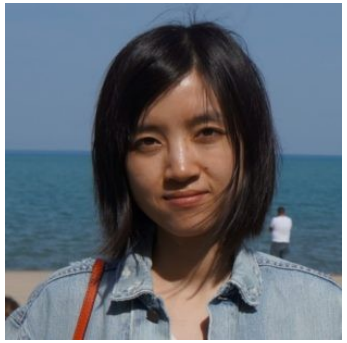


Ricardo Aravena

Cloud Native & AI Infrastructure

Open-source infrastructure
Kubernetes and AI/ML platforms

serverbooter.com



Rainie Li

Pinterest

Data processing infrastructure
Apache YuniKorn PMC member

Apache YuniKorn · rainieli

01

Unified scheduling

01 Unified scheduling

02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

Different paths. Shared infrastructure.

How do schedulers coordinate node access?

Different Workloads, Shared Infrastructure

Training, inference, and agent applications put different demands on the same cluster.

Training & RL

Coordinated compute



Start a complete group

Resources + coordination

Inference

Roles and complete replicas



Keep communication local

Roles + topology

Agent applications

Bursty, latency-sensitive requests



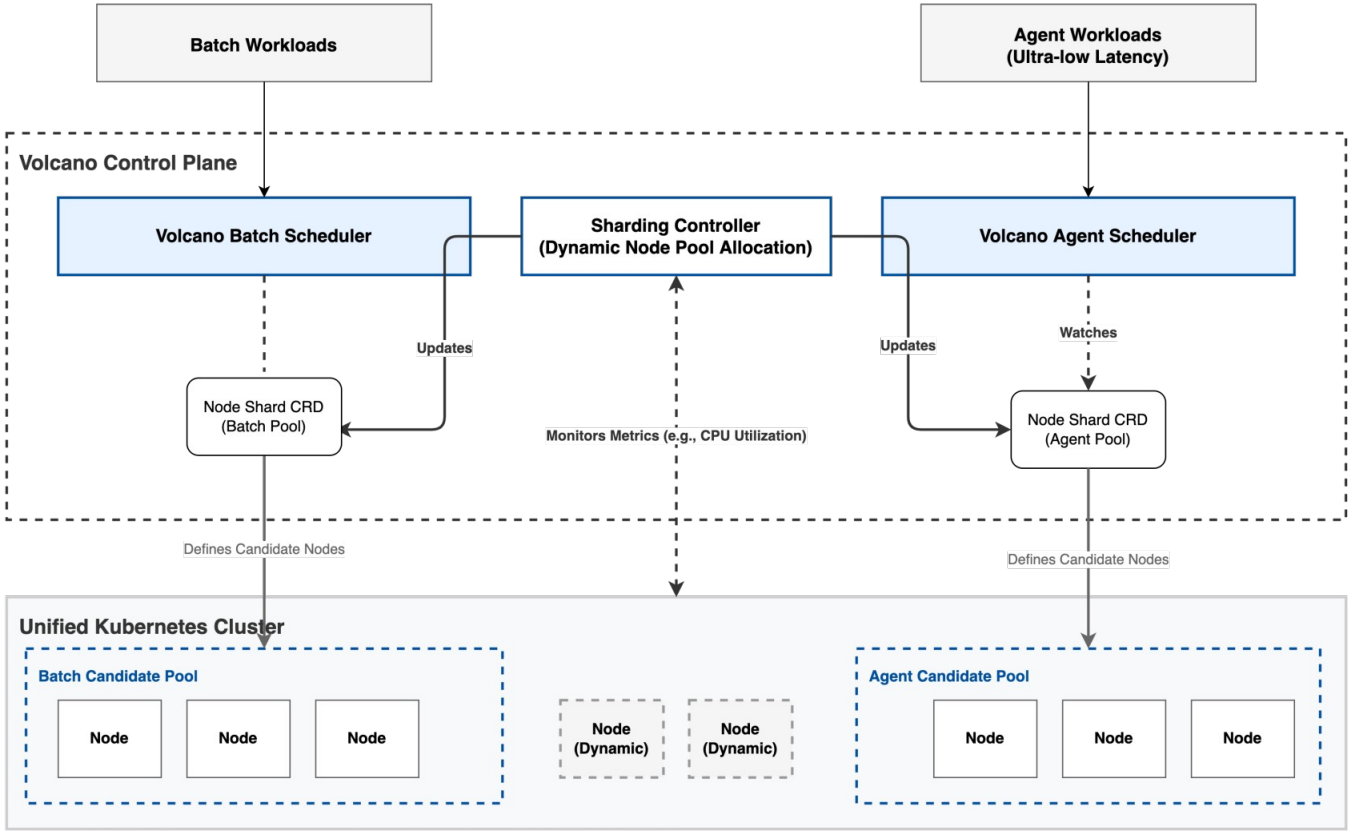
Respond to bursts

Runtime determines the unit

Shared cluster capacity; different requirements for coordination, locality, and latency.

A Coordinated Scheduling Framework

Specialized scheduling paths coordinate through NodeShard.



01 Workload paths

Batch-oriented scheduling and a Pod-oriented fast path.

02 Local coordination

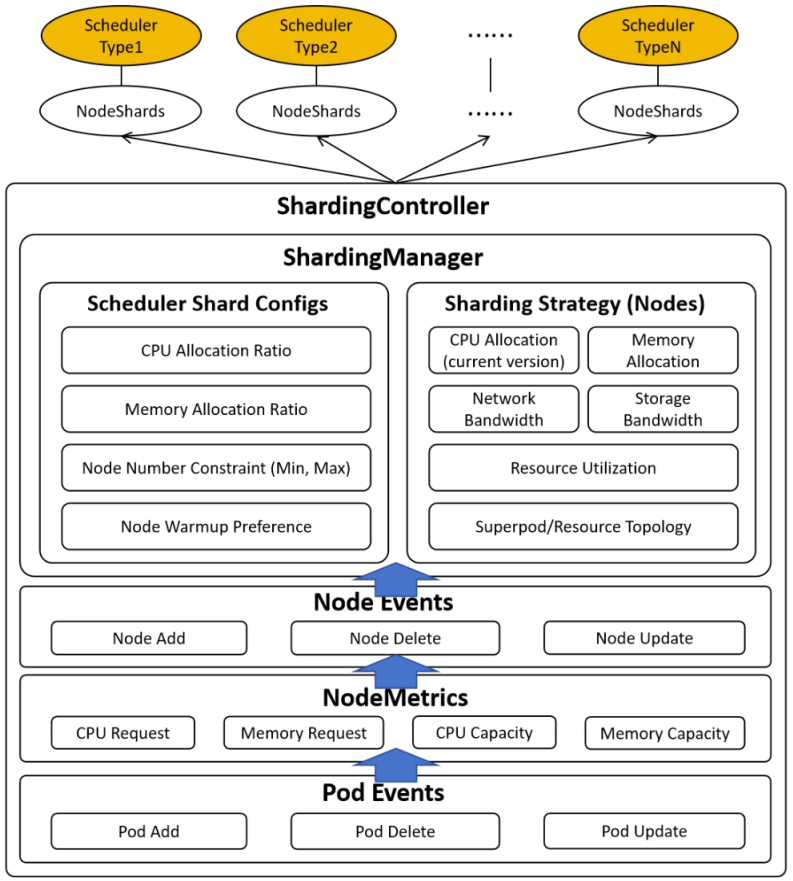
Parallel workers use Binder conflict checks.

03 Shared node access

NodeShard coordinates candidate-node pools.

NodeShard: Desired Nodes and Current Use

Changing a shard coordinates future placement; it does not migrate running Pods.



NodeShard · excerpt

```
kind: NodeShard
metadata:
  name: volcano
spec:
  nodesDesired: [node1, node2, node3]
status:
  nodesInUse: [node0, node1, node2]
  nodesToRemove: [node0]
  nodesToAdd: [node3]
```

Desired membership ≠ immediate use

Coordinators reconcile the handoff at scheduling boundaries.

02

Multi-level gang & network topology

From available nodes to runnable workloads.

What must run together, and where should it fit?

01 Unified scheduling

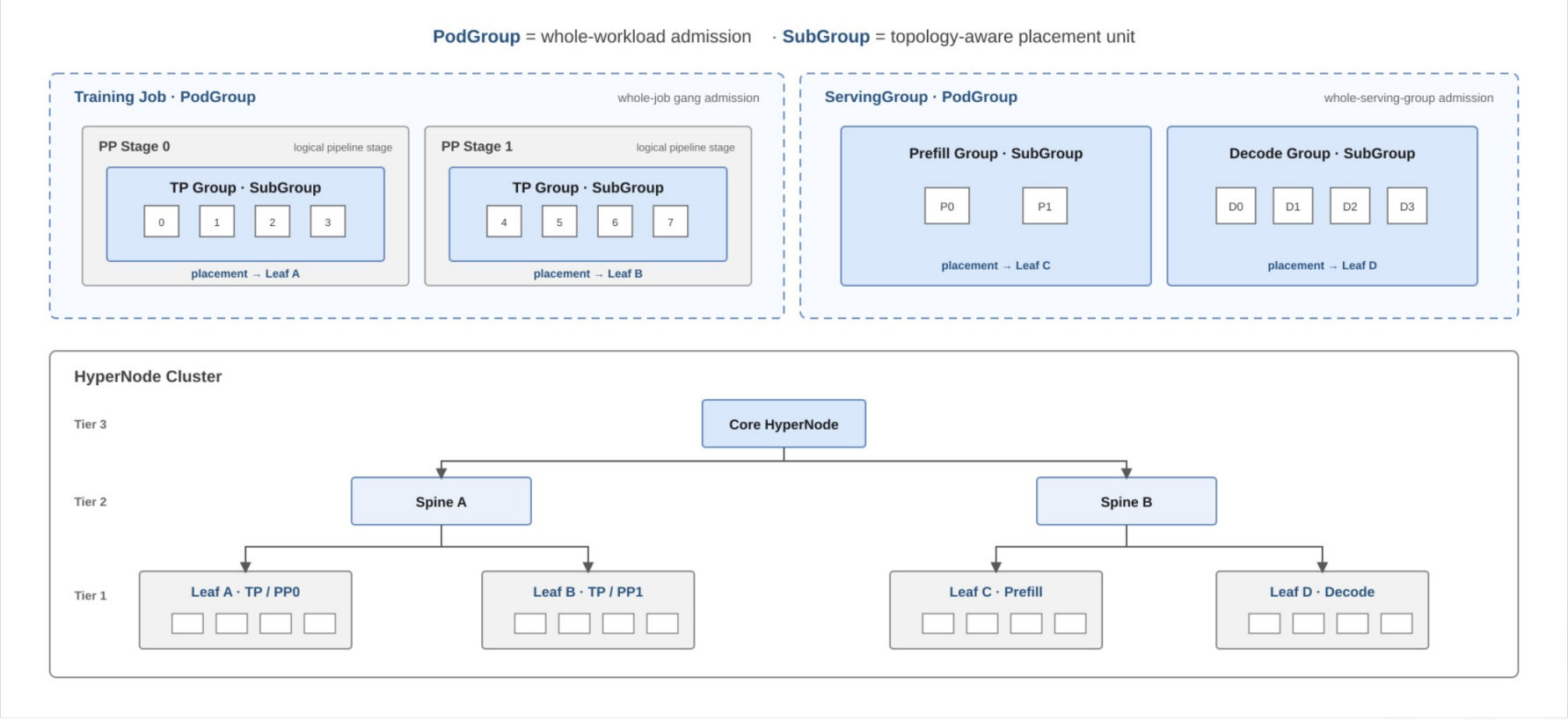
02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

PodGroup and SubGroup: Preserve Workload Structure

Gang requirements apply to the workload and to its complete subgroups.

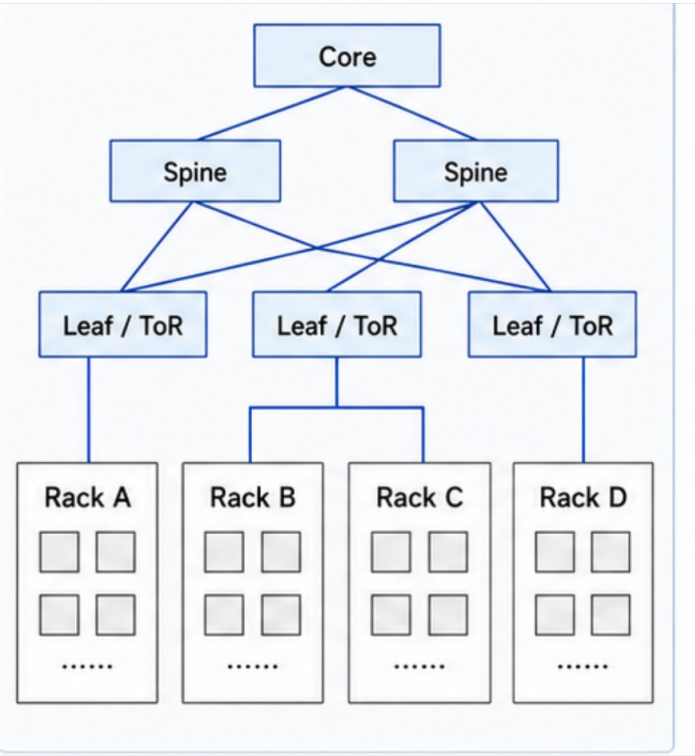


A SubGroup policy selects Pods; replica identity can split them into multiple SubGroups.

HyperNode: From Physical Fabric to Scheduling Domains

A parent HyperNode groups child HyperNodes; a leaf groups Kubernetes Nodes.

PHYSICAL FABRIC



Parent · HyperNode.spec

```
tier: 2
tierName: spine
members:
- type: HyperNode
  selector:
    exactMatch:
      name: rack-a
```

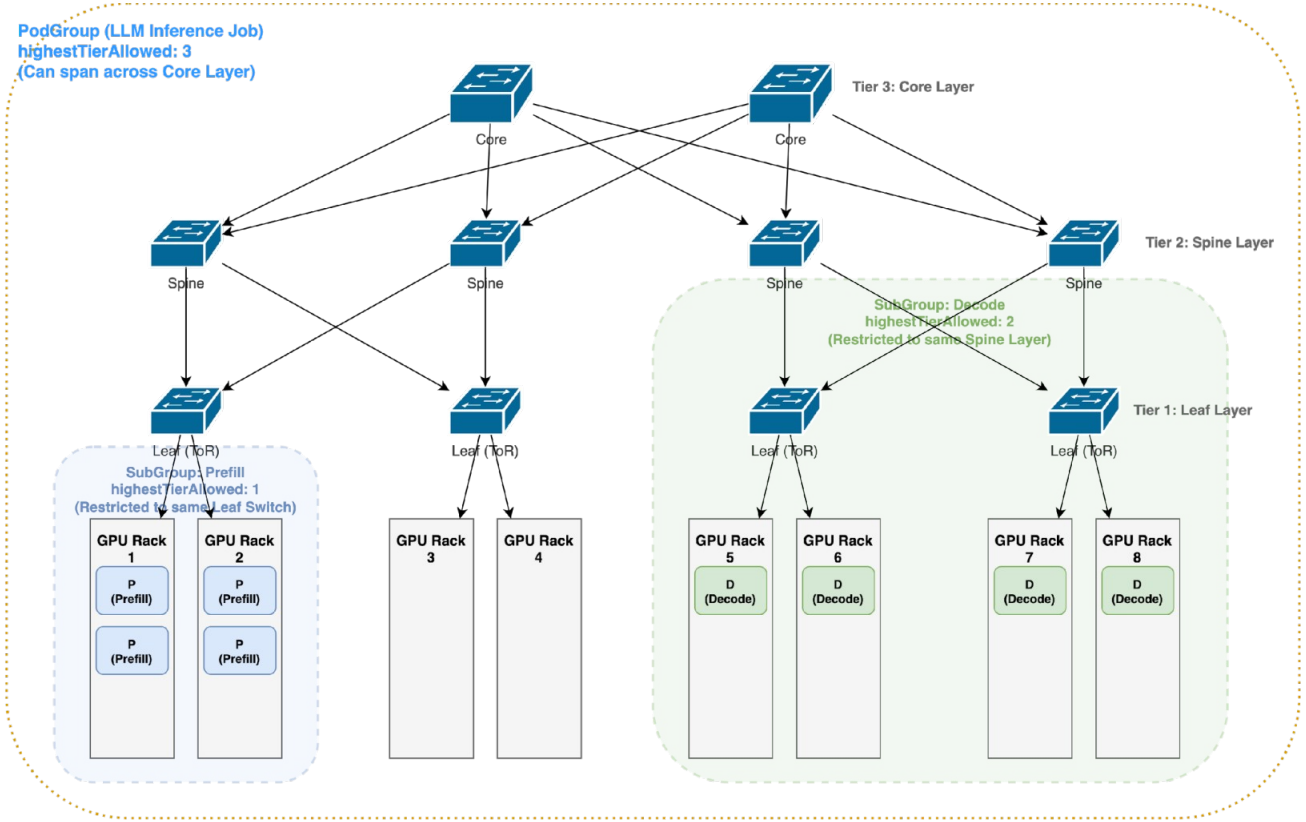
Leaf · HyperNode.spec

```
tier: 1
tierName: rack
members:
- type: Node
  selector:
    labelMatch:
      matchLabels:
        rack: rack-a
```

The same hierarchy is available through the standalone HyperNode component.

Place Groups at Different Topology Levels

An eight-Pod instance spans a larger domain; Prefill and Decode use different locality constraints.



Select each role's Pods in its policy; selectors are omitted here.

PodGroup.spec · key fields

```
minMember: 8
networkTopology:
  mode: hard
  highestTierAllowed: 3
subGroupPolicy:
- name: prefill
  subGroupSize: 4
  networkTopology:
    mode: hard
    highestTierAllowed: 1
- name: decode
  subGroupSize: 4
  networkTopology:
    mode: hard
    highestTierAllowed: 2
```

Keep Each Instance Together, Place Instances Apart

PodGroup anti-affinity separates service instances across HyperNode domains.



PodGroup · same rule on each instance

```yaml
metadata:
  labels: {app: model-a}
spec:
  networkTopology:
    mode: hard
    highestTierName: supernode
  topologyAffinity:
    podGroupAntiAffinity:
      required:
      - podGroupSelector:
          matchLabels: {app: model-a}
        topologyTierName: supernode
```

Inside an instance: locality. Between instances: separation. PodGroup anti-affinity is available in v1.16.

03

Ecosystem

01 Unified scheduling

02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

Workload controllers carry scheduling intent.

How do roles and replicas reach the scheduler?

Controllers Bring Workload Structure to Scheduling

Integration depth differs: group support and topology support are separate steps.

CONTROLLER INPUTS

Kthena

ModelServing / role replicas

Gang + topology available

KubeRay

RayCluster / multi-host replicas

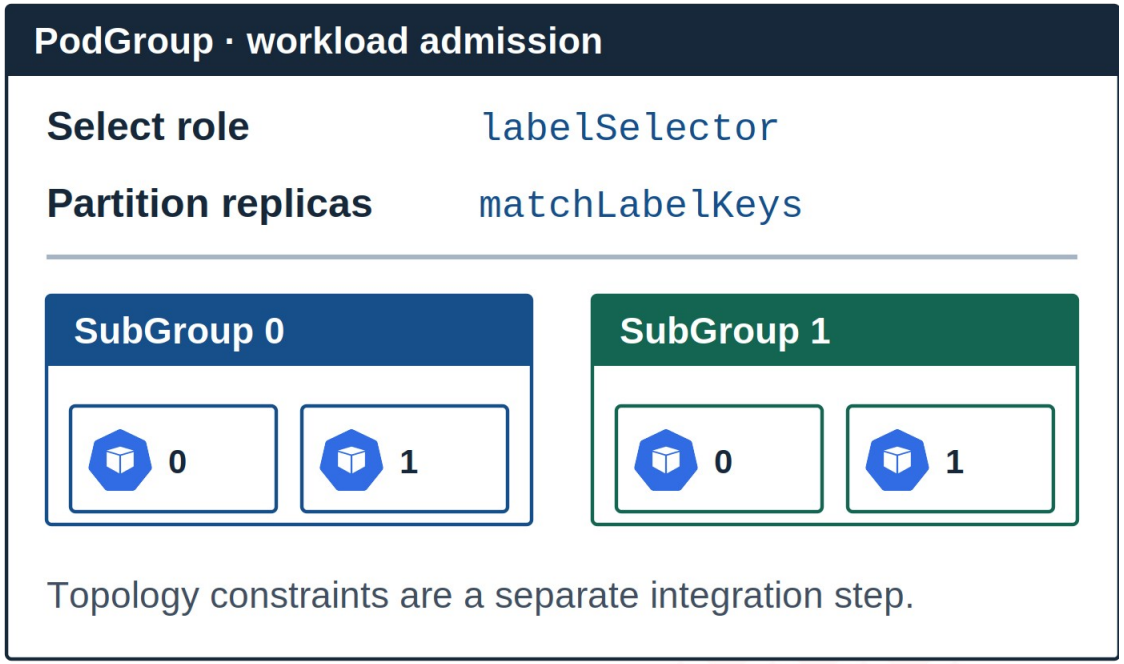
SubGroup merged · topology in progress

Grove

PodCliqueSet / scaling groups

SubGroup backend · topology mapping open

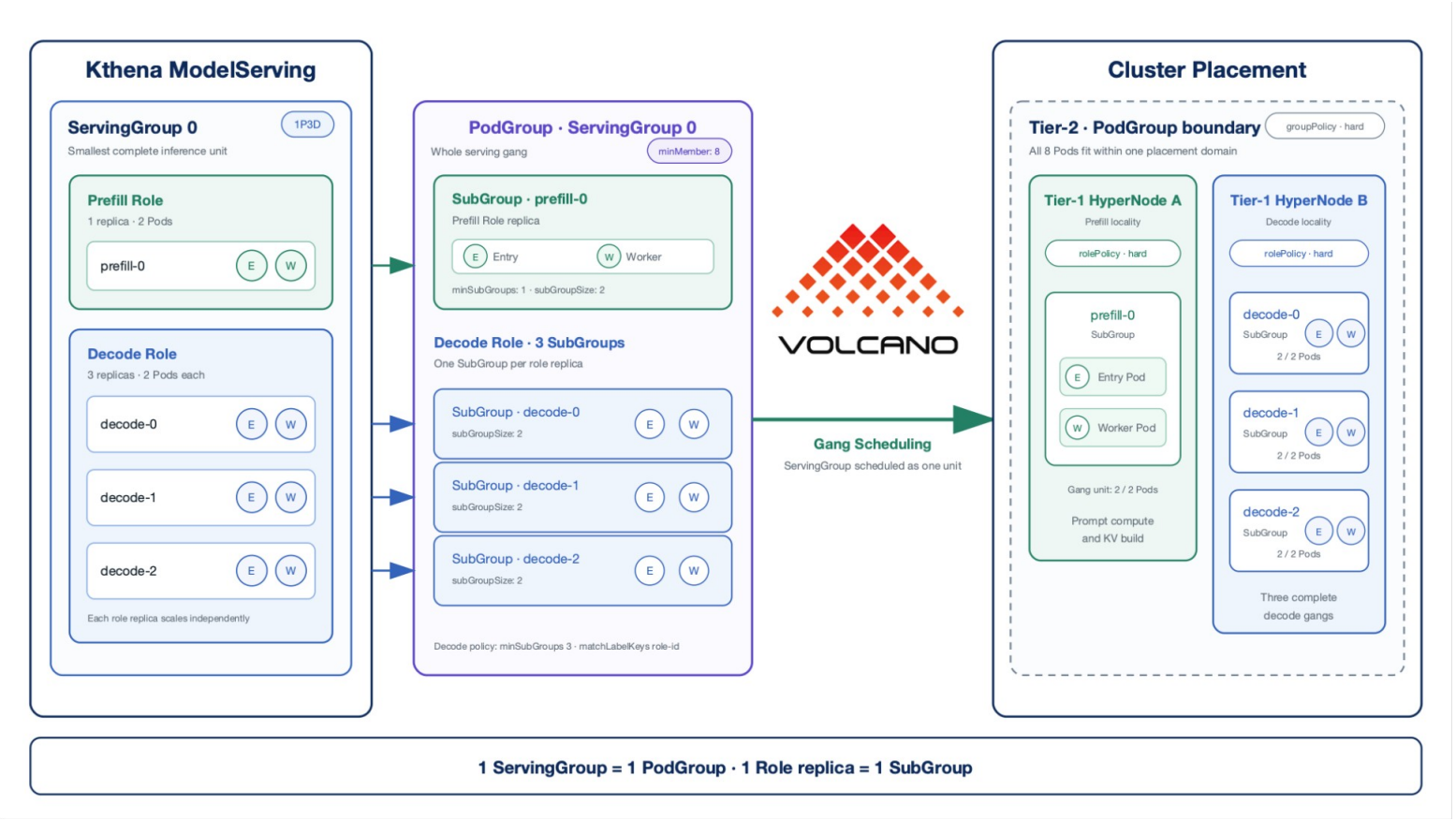
VOLCANO GROUPING MODEL



Kubeflow Trainer and LWS already integrate with Volcano; finer groups and locality are next.

Kthena: From Serving Roles to Scheduling Groups

One ServingGroup maps to one PodGroup; each role replica forms a SubGroup.



ModelServing.spec · excerpt

```
schedulerName: volcano
replicas: 1
template:
  gangPolicy:
    minRoleReplicas:
      prefill: 1
      decode: 3
  roles:
  - name: prefill
    replicas: 1
    workerReplicas: 1
  - name: decode
    replicas: 3
    workerReplicas: 1
```

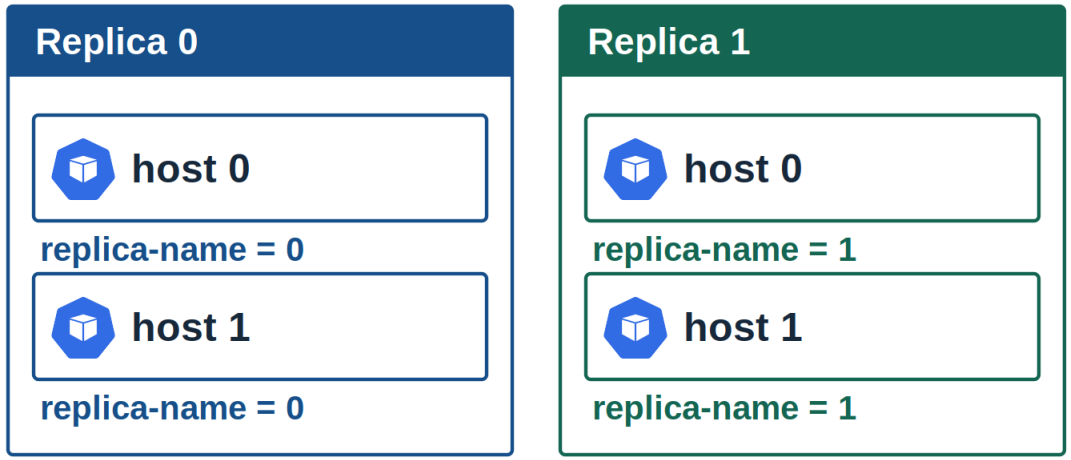
Pod templates omitted.

KubeRay: A Replica Can Span Multiple Pods

The replica label separates complete groups inside one WorkerGroup.

WorkerGroup · 2 replicas × 2 hosts

Each replica is one complete SubGroup.



Same replica label → same SubGroup

PodGroup.spec · generated grouping

```
subGroupPolicy:
- name: worker
  labelSelector:
    matchLabels: {ray.io/group: worker}
  matchLabelKeys:
  - ray.io/worker-group-replica-name
  minSubGroups: 2
  subGroupSize: 2
```

1. **Partition by the replica label**
2. **Require two complete, two-Pod groups**

Gang support is merged. Replica topology is in progress. The head has its own group.

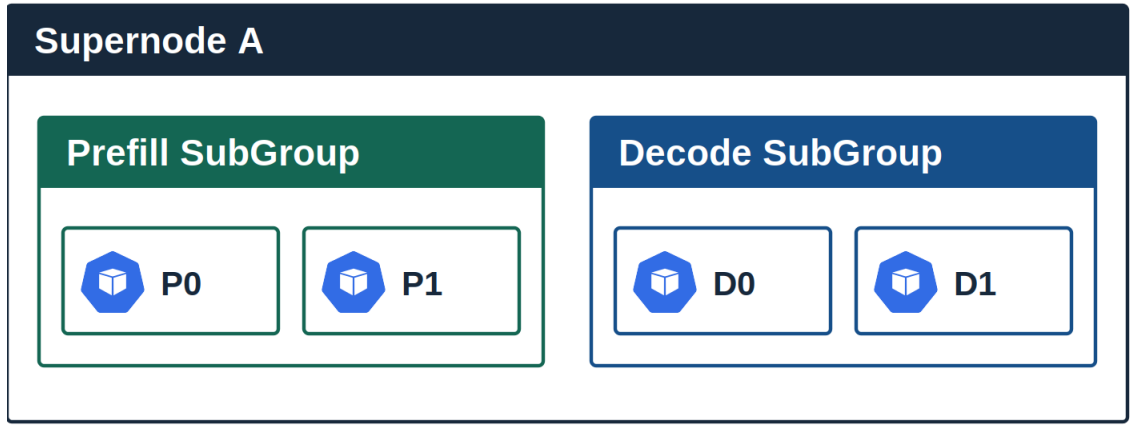
Next: Relationships Between SubGroups

Extend the placement model as controllers expose richer workload structure.

PROPOSED · SubGroup relationships within one PodGroup

01 Affinity

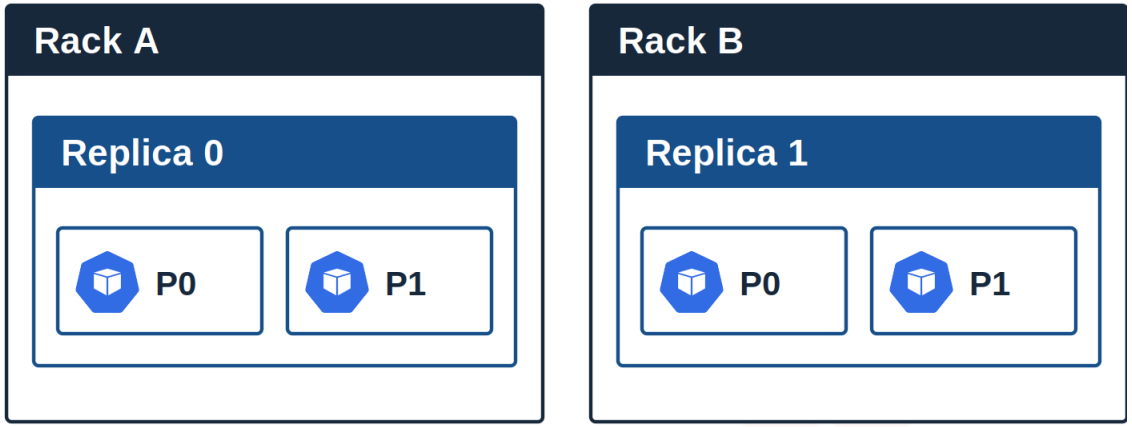
Cooperating groups share a domain



One shared topology boundary

02 Anti-affinity

Replicas occupy different domains



Distinct topology boundaries

Two placement intents, shown separately. SubGroup affinity / anti-affinity remains proposed.

04

Benchmark framework & progress

Measure behavior across workload shapes.

What can we reproduce, compare, and improve?

01 Unified scheduling

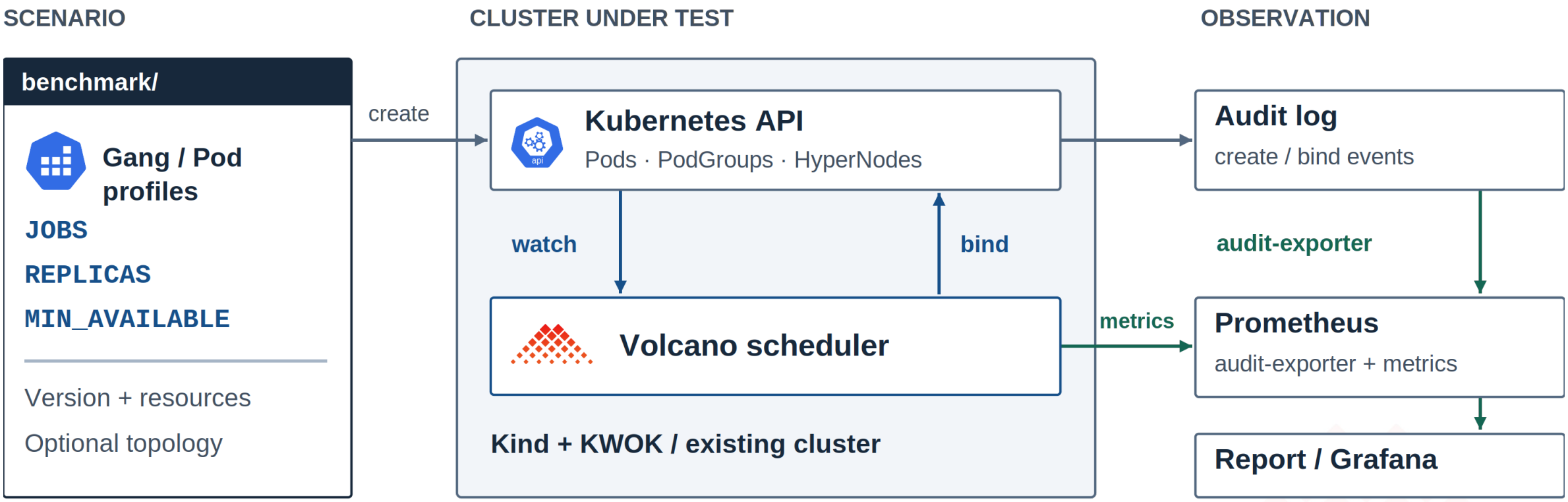
02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

A Repeatable Scheduling Benchmark Framework

Generate scenarios, observe API events, and retain comparable measurements.



The harness controls the scenario and records observations; KWOK does not execute GPU work.

Benchmark Progress and Ongoing Validation

The harness is available; testing and optimization continue across workload scenarios.

RUN A SCENARIO · benchmark/

```
make setup

make test-gang-env \
  JOBS=10 REPLICAS=100 \
  MIN_AVAILABLE=100
```

COLLECT THE EVIDENCE

`logs/kube-apiserver-audit.log`

`results/report-<timestamp>.json`

ONGOING VALIDATION

01 Vary one dimension

Workload size, grouping, or topology.

02 Keep the comparison fixed

Version, resources, and metric definitions.

03 Repeat and inspect

Retain raw events; investigate bottlenecks.

Available: scenario setup, Gang / Pod workloads, topology, metrics, and report collection.

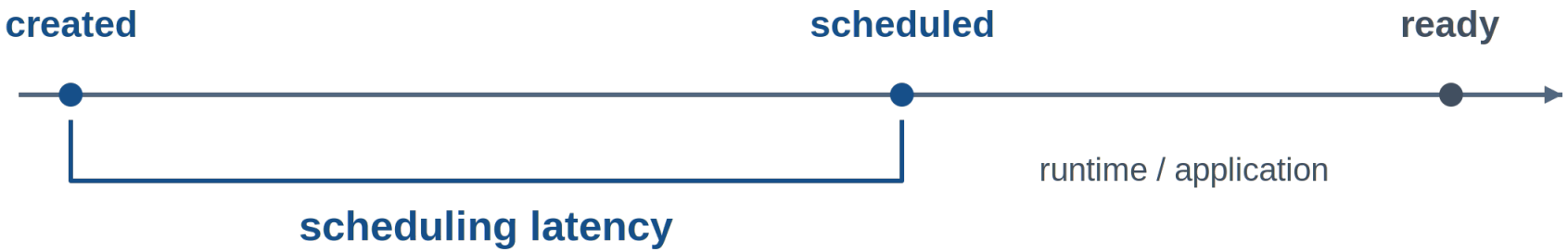
From Measurements to Reliable Comparisons

Keep scheduling latency distinct from application readiness.

MEASUREMENT DEFINITIONS · schematic, not benchmark results

Per-Pod latency

Created → scheduled



Scheduling throughput

Successful binds / window



Compare latency distributions and throughput with the same version, workload, resources, and window.

Framework, Workloads, Ecosystem, Evidence

Four connected pieces of the scheduling story.

01 **Coordinate the paths**

NodeShard + scheduler coordination

02 **Preserve workload structure**

Complete groups + topology + separation

03 **Carry intent through controllers**

Roles and replica identity reach scheduling

04 **Measure and keep improving**

Repeatable scenarios + comparable observations

Questions & Discussion

Workload shapes, integration feedback, and reproducible observations are welcome.

What must run together?

What should stay apart?

Bring a workload, a topology constraint, or a reproducible test.

github.com/volcano-sh/volcano

Roadmap and feedback · issue #6166